Ejercicio 1

```
def lee_resultados(fichero):
    res = []
    with open(fichero, encoding = 'utf-8') as f:
        lector = csv.reader(f, delimiter=";" )
        next(lector)
        for fecha, partido, result, score, competition, spectators in lector:
            fecha = parsea_fecha(fecha.strip())
            equipoloc, equipovis = parsea_partido(partido.strip())
            ganador = result.strip()
            golesloc, golesvis = parsea_score(score)
            competicion = competition.strip()
            espectadores = int(spectators.strip())
            res.append(Partido(fecha, equipoloc, equipovis, ganador, golesloc,
                golesvis, competicion, espectadores))
        return res

def parsea_fecha(cad):
    return datetime.strptime(cad, "%d/%m/%Y").date()

def parsea_partido(cad):
    lista = cad.split(" v ")
    return(lista[0].strip(), lista[1].strip())

def parsea_score(cad):
    lista = cad.split("-")
          return(int(lista[0].strip()), int(lista[1].strip()))
```

Módulo partidos_test

```
def test_lee_resultados(partidos):
    print(f"Se han leído {len(partidos)} registros")
    print("Los tres primeros registros son:")
    mostrar_enumerado(partidos[:3])
    print("Los tres últimos registros son:")
    mostrar_enumerado(partidos[-3:])

if __name__=="__main__":
    partidos = lee_resultados("../data/resultadosIsrael.csv")
    test_lee_resultados(partidos)
```

Ejercicio 2

```
def selecciones_enfrentadas_israel(partidos, n=3):
    num_par_por_competicion = Counter(partido.competicion for partido in partidos)
    mas_jugadas = sorted(num_par_por_competicion,
                                key = num_par_por_competicion.get, reverse = True)[:n]
    return {contrincante(p) for p in partidos if p.competicion in mas_jugadas}

def contrincante(partido):
    res = partido.equipovis
    if partido.equipovis == "Israel":
        res = partido.equipoloc
    return res
```

Módulo partidos_test

```
def test_selecciones_enfrentadas_israel(partidos, n=3):
    res = selecciones_enfrentadas_israel(partidos, n)
    print(f"Selecciones enfrentadas a Israel en las {n} competiciones más jugadas:")
    mostrar_enumerado(res)

if __name__=="__main__":
    partidos = lee_resultados("../data/resultadosIsrael.csv")
    test_selecciones_enfrentadas_israel(partidos, 3)
```

Ejercicio 3

```
def dicc_goles_por_fecha(lista, fecha1, fecha2):
    if fecha1 == None and fecha2 == None:
        lista_fil = lista
    elif fecha1 == None:
        lista_fil = [r for r in lista if r.fecha <= fecha2]
    elif fecha2 == None:
        lista_fil = [r for r in lista if r.fecha >= fecha1]
    else:
        lista_fil = [r for r in lista if r.fecha >= fecha1 and r.fecha <= fecha2]
    dicc = {}
    for r in lista_fil:
        if r.equipoloc == "Israel":
            dicc[r.fecha] = dicc.get(r.fecha, 0) + r.golesloc
        else:
            dicc[r.fecha] = dicc.get(r.fecha, 0) + r.golesvis
    return dicc

def lista_diferencias_goles(lista, fecha1=None, fecha2=None):
    dicc = dicc_goles_por_fecha(lista, fecha1, fecha2)
    lista_ord = sorted(dicc.items())
    lista_dif = list()
    for i in range(len(lista_ord) - 1):
        lista_dif.append(lista_ord[i+1][1] - lista_ord[i][1])
    return lista_dif
```

Solución alternativa:

```
def lista_diferencias_goles (partidos, f1=None, f2=None):
    lista_ord = sorted((p for p in partidos if en_fecha(p.fecha, f1, f2)),
                                                          key = lambda p: p.fecha)
    res = []
    for p1, p2 in zip(lista_ord, lista_ord[1:]):
        res.append(goles_israel(p2) - goles_israel(p1))
    return res

def en_fecha(fecha, fecha1, fecha2):
    res = True
    if fecha1 != None and fecha2 != None:
        res = fecha1 <= fecha <= fecha2
    elif fecha1 == None and fecha2 != None:
```

```
        res = fecha <= fecha2
    elif fecha1 != None and fecha2 == None:
        res = fecha1 <= fecha
    return res

def goles_israel(partido):
    res = partido.golesvis
    if partido.equipoloc == "Israel":
        res = partido.golesloc
    return res
```

Módulo partidos_test

```
def test_diferencia_goles (partidos, f1, f2):
    res = diferencia_goles (partidos, f1, f2)
    print(f"La diferencia de goles de los partidos de israel entre {f1} y {f2} es:")
    print(res)

if __name__=="__main__":
    partidos = lee_resultados("../data/resultadosIsrael.csv")
    f1 = datetime.datetime(2019, 9, 5).date()
    f2 = datetime.datetime(2020, 1, 5).date()
    test_diferencia_goles (partidos, f1, f2)
    test_diferencia_goles (partidos, f1, None)
    test_diferencia_goles (partidos, None, f2)
    test_diferencia_goles (partidos, None, None)
```

Ejercicio 4

```
def partidos_por_mes(partidos):
    c = Counter(partido.fecha.month for partido in partidos)
    return sorted(c.items())
```

Módulo partidos_test

```
def test_partidos_por_mes (partidos):
    res = partidos_por_mes (partidos)
    print("El número de partidos por mes es:")
    print(res)

if __name__=="__main__":
    partidos = lee_resultados("../data/resultadosIsrael.csv")
    test_partidos_por_mes (partidos, f1, f2)
```

Ejercicio 5

```
def partidos_mensuales_por_anyo(partidos):
    dicc = agrupar_por_anyo (partidos)
    return {anyo: partidos_por_mes(partidos) for anyo, partidos in dicc.items()}

def agrupar_por_anyo (partidos):
    res = dict()
```

```
    for partido in partidos:
        clave = partido.fecha.year
        if clave in res:
            res[clave].append(partido)
        else:
            res[clave] = [partido]
    return res
```

Módulo partidos_test

```
def test_partidos_mensuales_por_anyo(partidos):
    res = partidos_mensuales_por_anyo(partidos)
    print("El número de partidos mensuales por año es:")
          mostrar_enumerado(res.items())

if __name__=="__main__":
    partidos = lee_resultados("../data/resultadosIsrael.csv")
    test_partidos_mensuales_por_anyo(partidos)
```